

VQActFlow: Vector-Quantized Action Mode Steering for Multi-Task Robot Manipulation

Zhigen Zhao¹, Mark Leggiero¹, Yipu Chen¹, Haoran Liu¹, Yifan Wu¹, Huishu Xue¹, Sirui Zhan¹, and Ye Zhao¹

Abstract—Multi-task robot manipulation policies are challenging to learn from demonstration because a single network must select among qualitatively different action modes from a multimodal demonstration distribution, conditioned on language and visual context. A wrong mode selection means executing the wrong task or an action infeasible in the scene. Tokenizing continuous actions into a learned discrete codebook separates these modes at the representation level, offering structural advantages for multi-task learning. We propose VQActFlow, a multi-task manipulation policy that tokenizes action chunks and generates code sequences via Variational Flow Matching. VQActFlow maintains an explicit preference over action modes throughout generation. Inference-time guidance acts on this preference to steer mode commitment. We instantiate this with classifier-free guidance over language conditioning, which steers the policy toward the instructed action mode, and a learned codebook critic that supplies a complementary feasibility signal. We evaluate VQActFlow on three platforms: the LIBERO simulation benchmarks, a Unitree G1 humanoid performing whole-body pick-and-place, and an ALOHA-style bimanual platform performing contact-rich tasks. Across these benchmarks, VQActFlow outperforms both continuous and discrete baselines.

I. INTRODUCTION

Robot policies are increasingly expected to handle multiple manipulation tasks within a single model [1], [2], [3], [4], taking a language instruction and visual observation as input and dispatching to one of many learned behaviors at deployment time. Demonstration data for this setting is multimodal: the same observation recurs under different instructions and is followed by qualitatively different actions. A policy trained on such data must select the instructed action mode from this multimodal distribution, distinguishing modes that differ in target object, contact strategy, or bimanual coordination. When the observation is ambiguous, a wrong action mode selection is not a small numerical error but the robot carrying out a coherent behavior for the wrong task or performing the right task in a infeasible manner. The policy must therefore commit to the action mode consistent with the instruction and feasible in the scene.

Diffusion and flow matching policies [5], [6], [7] have become the prevailing paradigm for visuomotor manipulation, and classifier-free guidance (CFG) [8] and related steering methods [9], [10] are the standard tools for biasing them at inference. These policies generate in a continuous action

or latent space where action modes are not explicitly represented, so guidance reshapes a single continuous distribution rather than selecting among discrete action modes. Vector-quantized codebooks [11], [3], [12] instead learn a finite vocabulary of action primitives whose discrete bottleneck separates modes explicitly, exposing the structure the multi-task setting requires. Discrete Policy [12], the closest prior work to ours, tokenizes actions with a VQ-VAE but generates codes by diffusion in a continuous space, recovering discrete indices by nearest-neighbor lookup only at the end of sampling, abandoning the discrete structure during generation. The policy is left with no explicit representation of which mode it is committing to, which is exactly what the multi-task setting needs to resolve ambiguous observations toward the instructed task.

We propose VQActFlow, a multi-task manipulation policy that makes action mode commitment explicit: it maintains a preference over the discrete action modes throughout generation, so the mode commitment is represented at every step rather than recovered only at the end. VQActFlow generates action tokens via Variational Flow Matching (VFM) [13], [14], transporting Gaussian noise toward the embeddings of a frozen Vector-Quantized Variational Autoencoder (VQ-VAE) codebook under categorical supervision over code indices, so that it produces logits over codes at every integration step as an explicit preference over action modes. Inference-time guidance acts on this preference by redirecting probability toward the correct action mode, so the policy’s commitment can be steered at deployment rather than fixed at training time. We instantiate this guidance interface with two mechanisms. CFG amplifies the difference between task-conditioned and unconditional predictions to steer the policy toward the language-instructed action mode, and a learned codebook critic, trained via contrastive learning against structurally diverse negative examples, pushes the policy toward action modes feasible in the current scene. Both mechanisms act through the same interface and contribute complementary signals at each integration step.

Our contributions are as follows:

- 1) We propose VQActFlow, a multi-task manipulation policy that generates action tokens via Variational Flow Matching, maintaining an explicit preference over action modes throughout generation.
- 2) We show that guidance through this interface effectively steers action-mode selection at inference time, and introduce a novel contrastively-trained codebook

¹Institute for Robotics and Intelligent Machines (IRIM), Georgia Institute of Technology, Atlanta, GA 30332 {zhigen.zhao, mleggiero, ychen3302, hliu852, ywu3057, hxue37, sirui.zhan, yzhao301}@gatech.edu

critic that supplies a feasibility signal, operating on its own or alongside CFG.

- 3) We evaluate on the LIBERO benchmarks and on bimanual and humanoid hardware tasks, demonstrating improved performance over both continuous and discrete baselines.

II. RELATED WORK

Diffusion and flow-based robot policies. Diffusion models [15] generate samples by iteratively denoising Gaussian noise toward the data distribution. Flow matching [6] re-frames this as transporting noise along ODE trajectories defined by learned velocity fields. Diffusion Policy [5] applies the diffusion paradigm to robot action chunks, modeling multi-step action distributions via DDPM denoising and achieving strong visuomotor control on contact-rich manipulation tasks. DiffuseLoco [16] extends this paradigm to legged locomotion with receding-horizon control from offline multi-skill datasets, while Hybrid Diffusion [17] jointly produces symbolic and continuous plans via coupled discrete-continuous denoising. π_0 [7] builds a flow-matching policy with a VLM backbone for generalist manipulation. These methods operate in continuous action spaces, where guidance operates on continuous actions rather than discrete action modes. Discrete formulations of flow matching [18], [19] transport probability over categorical states directly, whereas Variational Flow Matching [13], [14] retains a continuous transport, casting it as variational inference over categorical posteriors. These methods have not been applied to robot policy learning. Discrete Policy [12], the closest prior work to ours, learns a VQ-VAE codebook of action primitives and generates code sequences via conditional latent diffusion in the embedding space.

Robot action tokenization. OpenVLA [20] discretizes actions via per-dimension scalar binning to reuse language model vocabularies, a simple scheme that sacrifices inter-dimension correlations. Discrete Policy [12] learns a VQ-VAE codebook over action sequences and shows that the learned codes cluster by task similarity, suggesting that VQ-VAE tokenization captures task-discriminative structure in multi-task action distributions. VQ-BeT [3] uses a residual VQ-VAE with hierarchical code prediction via classification heads and a continuous offset for refinement. FAST [21] applies DCT-based frequency compression followed by byte-pair encoding, while OAT [22] learns ordered tokens via finite scalar quantization and nested dropout.

Guidance for generative policies. Classifier guidance [9] steers diffusion sampling with a trained classifier’s gradients; CFG [8] avoids a separate classifier by interpolating conditional and unconditional scores. Guided Flows [23] extends guidance to flow matching with faster sampling than diffusion. In robotics, Diffuser [10] applies reward guidance for trajectory planning, SafeDiffuser [24] enforces safety constraints via control barrier functions, physics-informed diffusion [25] incorporates first-principles losses, and model-based diffusion [26] leverages learned dynamics models for trajectory optimization. VQActFlow differs from prior

robotics guidance in what guidance acts on: rather than steering continuous action distributions or trajectories, both CFG and the contrastively-trained codebook critic act on the policy’s categorical preference over discrete action tokens.

III. PRELIMINARIES

A. Vector-Quantized Variational Autoencoder

A VQ-VAE [11] compresses a continuous input into a sequence of indices into a learned codebook $\mathcal{C} = \{e_k\}_{k=1}^K$, where each $e_k \in \mathbb{R}^{d_e}$ is a codebook embedding. An encoder $\mathcal{E}$ produces a sequence of continuous latents $z \in \mathbb{R}^{L \times d_e}$, each of which is quantized to its nearest codebook entry:

$$k_l^* = \arg \min_k \|z^{(l)} - e_k\|_2, \quad z_q^{(l)} = e_{k_l^*}. \quad (1)$$

A decoder $\mathcal{D}$ reconstructs the input from the quantized embeddings z_q . The codebook is trained jointly with the encoder and decoder by combining a reconstruction loss with a commitment loss that pulls the encoder output toward its assigned codebook entry, with the codebook itself typically updated by exponential moving average [11], [27]. The result is a discrete bottleneck: any input is represented exactly by its sequence of L codebook indices.

B. Flow Matching

Flow matching [6] learns a generative model by training a neural network to predict a velocity field that transports samples from a simple source distribution p_0 (typically $\mathcal{N}(0, I)$) to a data distribution p_1 . Given a data sample $x_1 \sim p_1$, a conditional probability path $p_t(x | x_1)$ connects p_0 to x_1 , and is characterized by a conditional velocity field $u_t(x | x_1)$. A velocity network $v_\theta(x_t, t)$ is trained to regress this conditional velocity:

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, x_1, x_t \sim p_t(\cdot | x_1)} [\|v_\theta(x_t, t) - u_t(x_t | x_1)\|_2^2]. \quad (2)$$

A common choice is the optimal-transport conditional path, defined by sampling $x_0 \sim p_0$ and setting

$$x_t = (1 - t)x_0 + tx_1, \quad u_t(x_t | x_1) = x_1 - x_0, \quad (3)$$

which gives straight-line interpolants and a constant conditional velocity. At inference, the learned velocity field is integrated from $t=0$ to $t=1$, typically by Euler steps, to transport a fresh source sample $x_0 \sim p_0$ to a sample from the learned data distribution. Conditioning on context c is incorporated by passing c into $v_\theta(x_t, t, c)$ during both training and sampling.

C. Classifier-Free Guidance

CFG [8] steers a conditional generative model toward objectives more strongly aligned with the conditioning c without requiring an auxiliary classifier. During training, the conditioning is replaced by a learned null embedding $\emptyset$ with probability p_{drop} , so the same network v_θ learns to predict both the conditional output $v_\theta(x_t, t, c)$ and the unconditional output $v_\theta(x_t, t, \emptyset)$. At inference, the two outputs are linearly extrapolated to produce a guided output

$$\tilde{v} = v_\emptyset + w \cdot (v_c - v_\emptyset), \quad (4)$$

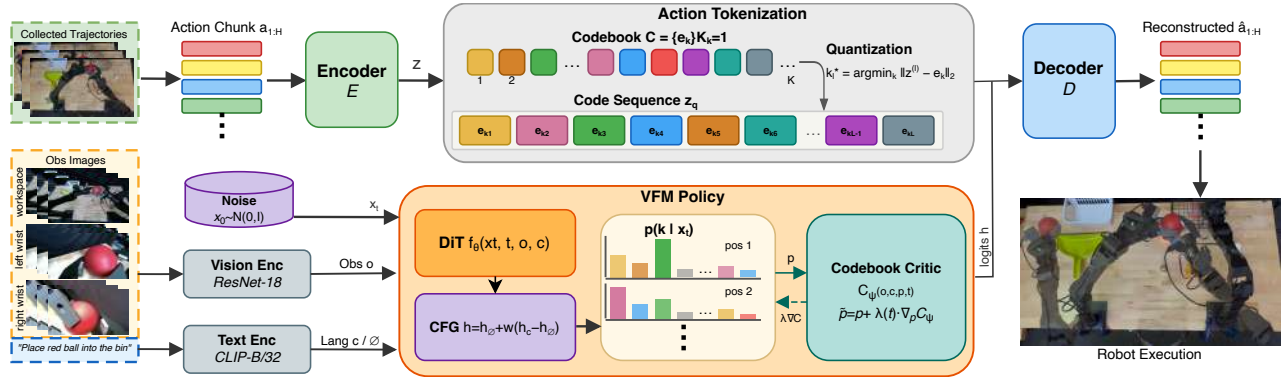


Fig. 1. VQActFlow framework. **Stage 1:** a VQ-VAE encoder tokenizes action chunks into discrete codebook indices, and a decoder reconstructs actions from the quantized embeddings. **Stage 2:** a VFM policy transports Gaussian noise toward codebook embeddings, maintaining an explicit preference over action modes that inference-time guidance steers, CFG toward the instructed task and a codebook critic toward feasible modes.

where $w \geq 1$ is the guidance weight. Setting $w=1$ recovers conditional sampling; $w > 1$ pushes the output further along the differential between conditional and unconditional predictions, sharpening the influence of c on the generated sample. CFG has become the standard inference-time control mechanism for diffusion and flow matching models.

IV. METHODS

VQActFlow generates manipulation actions as sequences of discrete tokens drawn from a VQ-VAE codebook (Sec. IV-A), with a VFM policy producing an explicit preference over codebook entries at every step of generation (Sec. IV-B). This preference provides a unified interface for inference-time guidance, on which CFG over language conditioning (Sec. IV-C) steers the policy toward the language-instructed action mode and a learned codebook critic (Sec. IV-D) pushes the policy toward modes feasible in the current scene. Fig. 1 summarizes the framework.

A. VQ-VAE Action Tokenization

We adopt the VQ-VAE framework from Sec. III-A to tokenize action chunks. Given an action chunk $a_{1:H} \in \mathbb{R}^{H \times d_a}$ spanning H timesteps with d_a action dimensions, a temporal encoder $\mathcal{E}$ maps the chunk to a sequence of continuous latents $z = \mathcal{E}(a_{1:H}) \in \mathbb{R}^{L \times d_e}$, where $L = H/2^S$ is the compressed sequence length determined by S temporal downsampling stages and d_e is the codebook embedding dimension. Each latent is quantized to its nearest codebook entry as in Eq. 1, and a temporal CNN decoder $\mathcal{D}$ reconstructs the action chunk from the quantized embeddings, so each output timestep depends on a local neighborhood of codes rather than on a single code in isolation. Each action chunk is thus represented by L indices into a codebook of size K , with $L \ll H$. After training, the VQ-VAE is frozen and used to produce supervision targets for the VFM policy in Sec. IV-B.

B. Variational Flow Matching Policy

To preserve the discrete structure of the codebook during generation, VFM trains the network on a cross-entropy over codebook indices rather than an L2 regression toward codebook embeddings, treating the K codes as competing

action modes and producing an explicit preference over them at every integration step. This preference is categorical but carried within a continuous transport. A fully discrete process [19] over codes would represent the modes just as explicitly, but expose no continuous quantity for guidance to adjust during integration.

Training. Given a demonstration sample, the frozen VQ-VAE encoder (Sec. IV-A) produces target embeddings $x_1 = z_q \in \mathbb{R}^{L \times d_e}$ and ground-truth indices $\{k_l^*\}_{l=1}^L$. We sample source noise $x_0 \sim \mathcal{N}(0, I) \in \mathbb{R}^{L \times d_e}$ and a timestep $t \sim \mathcal{U}(0, 1)$, and construct the optimal-transport interpolant $x_t = (1-t)x_0 + tx_1$ from Eq. 3. The policy network f_θ processes x_t conditioned on t , an observation encoding o , and language conditioning c , producing logits over the codebook at every position:

$$h^{(l)} = f_\theta(x_t, t, o, c)^{(l)} \in \mathbb{R}^K, \quad l = 1, \dots, L. \quad (5)$$

Training minimizes the cross-entropy between the predicted categorical posterior and the ground-truth codebook indices:

$$\mathcal{L}_{\text{VFM}} = - \sum_{l=1}^L \left(h_{k_l^*}^{(l)} - \log \sum_{j=1}^K \exp h_j^{(l)} \right). \quad (6)$$

Unlike Eq. 2, the loss provides no direct gradient on the velocity field; the velocity is instead derived from the categorical posterior at inference. This objective trains the network to discriminate the correct action mode at each position from the $K-1$ alternatives, producing a sharp preference when the demonstration is unambiguous and a diffuse one otherwise.

Inference. At each integration step, we convert the categorical output to a velocity field via the posterior-weighted mean of the codebook. Let $p_k^{(l)} = \exp(h_k^{(l)}) / \sum_j \exp(h_j^{(l)})$. The posterior mean in embedding space is

$$\mu_1^{(l)} = \sum_{k=1}^K p_k^{(l)} \cdot e_k, \quad (7)$$

which we use as the predicted endpoint to compute the OT velocity

$$v_\theta(x_t, t) = \frac{\mu_1 - x_t}{1 - t}. \quad (8)$$

The velocity points toward the codebook-weighted average of the action modes currently favored by the policy, with $p_k^{(l)}$ controlling the relative weight on each candidate. As the policy concentrates probability on a single mode during integration, this average becomes dominated by the embedding of the favored mode. Starting from $x_0 \sim \mathcal{N}(0, I)$, we integrate $\dot{x} = v_\theta(x, t)$ from $t=0$ to $t=1$ using Euler steps. The terminal state is quantized position-wise, $k_i^* = \arg \min_k \|x_1^{(l)} - e_k\|_2$, and the resulting code sequence is decoded to actions $a_{1:H} = \mathcal{D}(e_{k_1^*}, \dots, e_{k_L^*})$.

Architecture. The policy network f_θ is a Diffusion Transformer (DiT) [28] backbone with cross-attention for visual and text conditioning and adaptive layer normalization for the flow-matching timestep embedding. The visual encoder is a ResNet-18 [29] and the text encoder is a frozen CLIP-B/32 [30].

C. Classifier-Free Guidance for Language Conditioning

We apply CFG (Sec. III-C) to the language conditioning c , with the network producing conditional logits $h_c = f_\theta(x_t, t, o, c)$ and unconditional logits $h_\emptyset = f_\theta(x_t, t, o, \emptyset)$. At each integration step, the two are linearly extrapolated on the logits rather than on the velocity:

$$\tilde{h} = h_\emptyset + w \cdot (h_c - h_\emptyset), \quad (9)$$

and the guided probabilities are computed as

$$\tilde{p}_k^{(l)} = \frac{\exp(\tilde{h}_k^{(l)})}{\sum_{j=1}^K \exp(\tilde{h}_j^{(l)})}, \quad (10)$$

which replace p in Eq. 7 to compute the posterior mean and velocity. The guided distribution biases the policy’s mode preference toward codes consistent with the language conditioning, effectively sharpening which action mode the policy commits to under ambiguous observations.

D. Codebook Critic for Scene-Conditioned Feasibility

The codebook critic supplies a learned feasibility signal that biases the policy’s mode preference toward actions feasible in the current scene. It operates on the categorical distribution over codes rather than on decoded actions, so each guidance step requires only one forward and backward pass through a small network, with no codebook decoding in the inner loop. Because the critic is structured independently of the policy, its objective can be adapted to different feasibility criteria without retraining the policy itself, and it is trained on demonstration data alone, with no simulator rollouts or policy samples required.

Guided velocity. During ODE integration, the critic gradient adjusts the policy’s preference over action modes:

$$\tilde{p} = \text{Proj}_\Delta[p + \lambda(t) \cdot \Pi_T[\nabla_p C_\psi(o, c, p, t)]]. \quad (11)$$

Here $p \in \mathbb{R}^{L \times K}$ is the current categorical distribution over codebook entries, $\Pi_T[g] = g - \bar{g}$ removes the component-wise mean to keep the gradient tangent to the simplex, and

Proj_Δ projects onto the probability simplex. The guidance weight adapts to the entropy of the current distribution:

$$\lambda(t) = \lambda_{\max} \left(1 - \frac{H(p)}{\log K} \right), \quad (12)$$

where $H(p) = -\sum_k p_k \log p_k$. Early in integration, p is near-uniform and reflects the noise distribution rather than the data; guidance at this stage would push toward arbitrary codes. As p concentrates over the course of integration, the distribution carries meaningful information about likely code selection, and guidance can meaningfully redirect it. The schedule therefore couples guidance strength to distributional confidence: $\lambda(t)$ is small when p is diffuse and strengthens as p crystallizes.

Critic architecture and training. The critic $C_\psi(o, c, p, t)$ is a transformer encoder that maps a visual observation o , language conditioning c , sequence of L simplex positions $p \in \mathbb{R}^{L \times K}$, and flow timestep t to a scalar feasibility score. The visual and language inputs are fused by an MLP into a conditioning token that is prepended to the code sequence. Each $p^{(l)}$ is linearly projected to d_{model} and combined with a learned positional embedding and a sinusoidal encoding of t . A pre-norm transformer encoder processes the sequence, and a mean-pooled representation is projected to a scalar by an MLP head.

Training uses an InfoNCE loss [31] that contrasts a positive (o, c, p) triple against negatives constructed to violate distinct components of trajectory validity:

- *Temporal shuffle*: permutes the order of the code sequence.
- *Random code replacement*: replaces a fraction of positions with uniformly sampled codebook entries.
- *Wrong observation*: pairs a valid code sequence with a mismatched observation from another sample in the batch.
- *Cross-demonstration swap*: replaces the code sequence with one drawn from a different demonstration.
- *In-episode temporal shift*: replaces the code sequence with one from the same episode at a different timestep.
- *Wrong language, correct vision*: permutes the language conditioning across the batch while keeping vision and codes paired.
- *Wrong vision, correct language*: permutes the visual conditioning while keeping language and codes paired.

Together these negatives expose the critic to violations along distinct dimensions of action validity, including temporal coherence, observation grounding, and language grounding, so that the learned feasibility score reflects all three rather than any single one. This allows the critic to steer the policy’s mode preference along axes that CFG does not: temporal coherence of the action sequence, observation grounding, and scene-conditioned feasibility.

All samples are noised to random flow timesteps $t \sim \mathcal{U}(t_{\min}, t_{\max})$ via the interpolation $x_t = (1-t)/K + t \cdot x_{\text{clean}}$, exposing the critic to a range of distributional sharpness.

V. EXPERIMENTS

We evaluate VQActFlow policy in simulation on the LIBERO benchmark suites [32] and on hardware with bi-manual manipulation tasks on a set of ALOHA-style arms and a Unitree G1 humanoid robot. We evaluate VQActFlow against three claims. First, the categorical interface enables effective inference-time guidance for action-mode selection (Sec. V-A, Sec. V-B, Sec. V-C). Second, the contrastively-trained codebook critic supplies a useful feasibility signal, operating independently or alongside CFG (Sec. V-B, Sec. V-D). Third, VQActFlow achieves the highest success rates among comparable continuous and discrete baselines across LIBERO-90, bimanual hardware, and humanoid hardware (Sec. V-B, Sec. V-D, Sec. V-C).

Throughout the experiments, we use a flat VQ-VAE codebook of size $K=512$ with $S=2$ temporal downsampling stages. The VFM policy backbone is a DiT with 12 layers, hidden size $d_{\text{model}}=1024$, and 8 attention heads. The codebook critic is a smaller transformer with 3 layers, hidden size 256, and 4 attention heads. Both the policy and critic use a frozen CLIP-B/32 [30] text encoder; the policy and critic visual encoder is shared and held frozen during critic training. The implementation uses the `flow_matching` [6] and `LeRobot` [33] libraries. All baseline policies are trained and evaluated under matched protocol: identical vision and text encoders, training dataset, number of training steps, and initialization seeds for simulation benchmarks. The Discrete Policy [12] baseline is implemented with shared VQ-VAE weights as VQActFlow, as well as identical DiT backbone.

A. CFG Evaluation on LIBERO-Goal

LIBERO-Goal tests whether guidance through the categorical interface is more effective than guidance through a continuous proxy. We sweep the CFG weight $w \in \{1, 2, \dots, 8\}$ on LIBERO-Goal with 20 rollouts per task and compare against Discrete Policy under matched conditions. VQActFlow improves from 73.0% unguided to 81.0% at $w=4$, then degrades monotonically to 63.0% at $w=8$ as per-task variance widens, as seen in Fig. 2. Discrete Policy peaks at $w=6$ with 61.5%, requiring stronger guidance to reach a substantially lower ceiling. VQActFlow’s tighter standard deviation band further indicates that the gain is consistent across tasks rather than concentrated on a few easy ones.

The aggregate response masks substantial per-task heterogeneity (Fig. 3). Tasks with significant unguided headroom gain the most from CFG: “open the top drawer and put the bowl inside” ($40 \rightarrow 65\%$ at $w=4$), “put the wine bottle on top of the cabinet” ($65 \rightarrow 85\%$), and “put the cream cheese in the bowl” ($60 \rightarrow 80\%$). Near-ceiling tasks regress: “open the middle drawer” drops from 90% to 75% at $w=4$. Rollout inspection in Fig. 4 reveals two distinct failure regimes. At low w , the policy executes coherent actions on the wrong object, indicating that the unconditional marginal $h_{\emptyset}$ is insufficiently suppressed. At high w , the policy reaches the correct object but misses the grasp, suggesting that aggressive amplification of $(h_c - h_{\emptyset})$ degrades execution

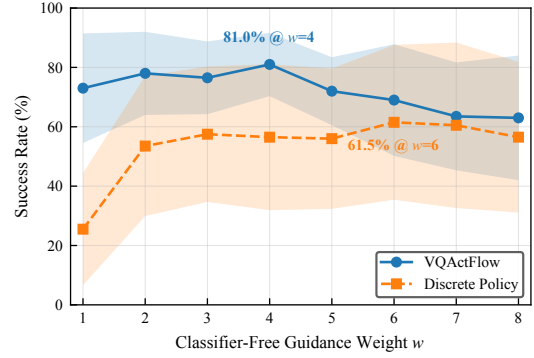


Fig. 2. LIBERO-Goal success rate vs. CFG weight. VQActFlow peaks at $w=4$ (81.0%) then degrades at higher weights. Discrete Policy [12] is shown for reference under matched conditions. Shaded: ± 1 std across 10 tasks.

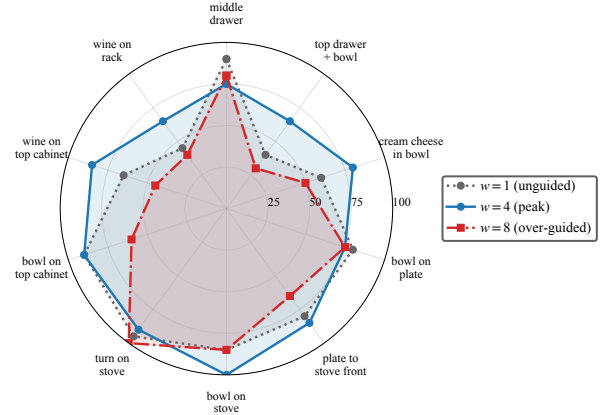


Fig. 3. Per-task LIBERO-Goal success at three CFG weights. The $w=4$ polygon expands on tasks with substantial unguided headroom and contracts on near-ceiling tasks.

precision. Optimal w thus trades off suppression of wrong-task modes against degradation of execution.

B. Multi-task Scaling on LIBERO-90

LIBERO-90 evaluates VQActFlow on 90 multi-task scenarios that test both the guidance interface at scale and the contribution of the codebook critic. All baselines are trained from scratch under matched conditions to isolate the action-generation mechanism, so vision-language-action (VLA) models [20], [7] with large pretraining datasets fall outside this comparison. We report VQActFlow with CFG alone, critic guidance alone, and both combined, along with baselines, as seen in Table I. VQActFlow with $w=2$ and $\lambda=1.0$ reaches 80.5%, surpassing Continuous Flow Matching DiT (CFM), MT-ACT [1], and all VQ-based baselines.

The critic and CFG provide complementary improvements. Critic guidance alone adds +2.0% at $w=1$, confirming that the contrastive feasibility signal carries information that the policy’s own task-conditional logits do not encode. Stacking critic guidance on top of CFG at $w=2$ contributes a further +2.9%. The composed configuration produces the best result on LIBERO-90, indicating that the two guidance methods provide largely non-redundant signals: CFG sharpens task identity through the contrast between h_c and $h_{\emptyset}$, while the critic injects a feasibility gradient learned against a broader set of validity violations. Among baselines with VQ-

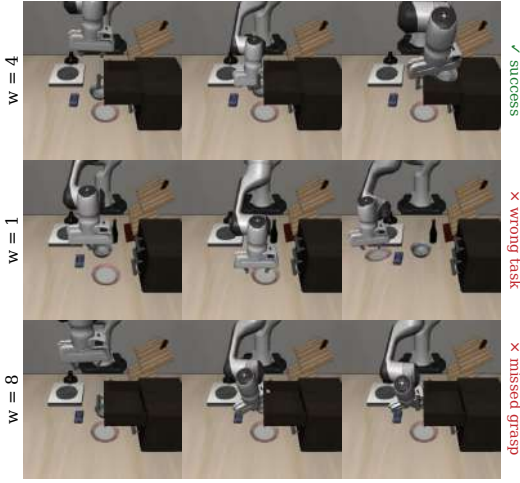


Fig. 4. CFG weight effects on the LIBERO task “open the top drawer and put the bowl inside.” Top: success ($w=4$). Middle: wrong-task failure ($w=1$). Bottom: missed-grasp failure ($w=8$).

TABLE I

LIBERO-90 SUCCESS RATE (%) FOR DIFFERENT POLICIES.

Method	Type	CFG w	Critic λ	Success
MT-ACT [1]	Continuous	–	–	72.4
CFM	Continuous	–	–	77.2
VQ-BeT [3]	VQ-based	–	–	24.1
Discrete Policy [12]	VQ-based	1.0	–	49.4
Discrete Policy	VQ-based	2.0	–	60.3
VQActFlow	VQ-based	1.0	0.0	72.3
VQActFlow	VQ-based	1.0	1.0	74.3
VQActFlow	VQ-based	2.0	0.0	77.6
VQActFlow	VQ-based	2.0	1.0	80.5

based latent action spaces, VQ-BeT achieves only 24.1%, and Discrete Policy with CFG trails VQActFlow at matched guidance weight by 17.3%, confirming that the categorical-versus-embedding training-objective gap persists on a harder benchmark.

Table II ablates the codebook size K on LIBERO-90 with guidance disabled. Success peaks at $K=512$: smaller codebooks limit vocabulary granularity, while larger ones enlarge the per-position classification space, making the categorical prediction harder for the policy to learn.

C. Humanoid Whole-Body Manipulation

We evaluate VQActFlow on a Unitree G1 humanoid equipped with two Inspire RH56DFTP dexterous hands, operated under whole-body control, shown in Fig. 5(a). The proprioceptive state is 46-dimensional and the action space is 47-dimensional, with egocentric vision from a Intel Realsense D435i head camera at 424×240 . Four pick-and-place tasks place one of a red ball, a cup, a bottle, or a medicine bottle into a shared bin (Fig. 5(b–e)). The four tasks share a workspace and differ only in target object and language instruction, isolating language-conditioned task disambiguation as the sole source of cross-task variation. We collect 140 VR-teleoperated demonstrations per task using TWIST2 [34] and roll out 20 trials per task at $w=1$ and $w=6$, scoring each as success, missed grasp (correct target, failed execution), or wrong task (incorrect target).

CFG raises aggregate success from 23.8% to 57.5% (Table III). A more detailed breakdown in Fig. 6 shows

TABLE II
ABLATION FOR CODEBOOK SIZES WITH LIBERO-90 (%).

Cookbook size	128	256	512	1024
Success rate	62.6	65.6	72.3	70.1

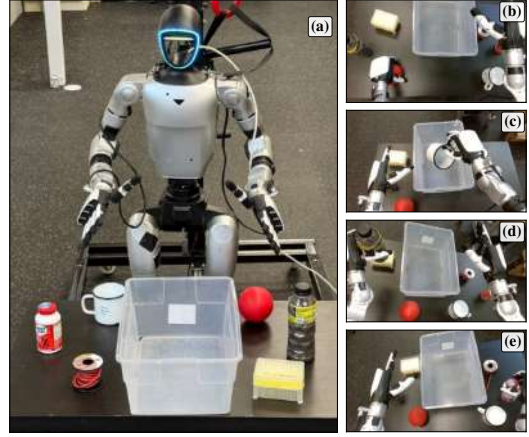


Fig. 5. G1 humanoid experimental setup and evaluation tasks. (a) Multitask tabletop scene. (b) Place the red ball into the box. (c) Place the cup into the box. (d) Place the bottle into the box. (e) Place the medicine bottle into the box.

all four tasks improving with CFG: red ball (+15%), cup (+45%), bottle (+60%), and medicine (+15%). The gain is driven almost entirely by the correction of wrong-task errors, which fall from 41.3% to 2.5%, while the missed-grasp rate is approximately unchanged. This asymmetric gain reflects CFG’s training objective: conditioning dropout teaches the policy to encode task identity in $h_c - h_\emptyset$, so guidance improves task disambiguation but not execution quality. The result mirrors the failure-mode dichotomy in simulation.

D. Bimanual Manipulation

We evaluate VQActFlow on a bimanual platform of two ALOHA-style arms, as seen in Fig. 7. Visual observations streams from three Intel RealSense D405i cameras: one wrist-mounted camera per arm and one fixed workspace camera. We design four contact-rich manipulation tasks (Fig. 8): (1) sweeping a battery into a dust pan, (2) placing a rubber duck into a white bin, (3) inserting a cylinder into a hole, and (4) placing a red ball into a black bin. Tasks 1, 2, and 4 require coordinated use of both arms, while 3 is a single-arm peg-in-hole task that challenges insertion accuracy. We collect 75 demonstration episodes per task via VR teleoperation using XRRoboToolkit [35].

VQActFlow is compared against Discrete Policy with 20 rollouts per task. The result is shown in Table IV. A CFM baseline with the same DiT backbone is excluded from the table because it failed to complete any task; we examine this failure at the end of the section.

CFG boosts VQActFlow more than Discrete Policy. Without guidance, VQActFlow and Discrete Policy achieve similar average success rates, indicating that categorical cross-entropy supervision and embedding regression produce comparable baseline performance. Raising w to 4 lifts VQActFlow to 73.8% but Discrete Policy only to 61.3%, reproducing the LIBERO-Goal pattern in Sec. V-A on a harder

TABLE III
AGGREGATED G1 HARDWARE POLICY SUCCESS RATE (%).

CFG weight	Success	Missed grasp	Wrong task
$w=1$	23.8	35.0	41.3
$w=6$	57.5	40.0	2.5

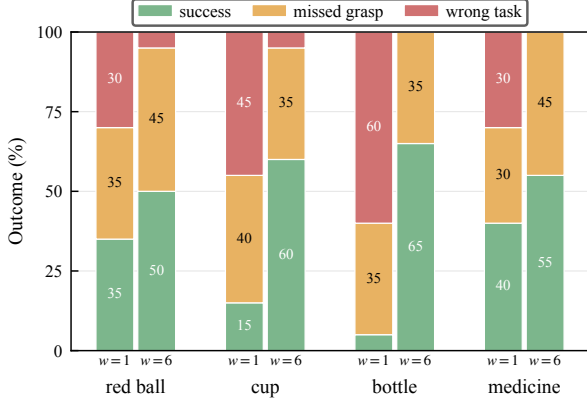


Fig. 6. G1 rollout outcomes across 4 pick-and-place tasks (20 trials each) at $w=1$ vs. $w=6$.

hardware benchmark.

Critic guidance independently improves performance. Holding $w=1$ and adding critic guidance at $\lambda=1$ raises VQActFlow from 45.0% to 60.0%, with the largest gain on the cylinder peg-in-hole task. The critic’s training signal contrasts valid trajectories against negatives that violate temporal coherence, code distribution, observation grounding, and language grounding, so the gradient amplifies feasibility along axes that CFG’s task-only training signal does not.

Combining $w=4$ and $\lambda=1$ yields the best configuration at 77.5%. The improvement over CFG-only concentrates on the cylinder and duck tasks, where execution precision is the dominant remaining failure mode after task identity is resolved. The two interfaces target largely non-redundant signals: CFG amplifies the task-conditional differential it is trained to encode, while the critic amplifies the broader feasibility signal its negatives induce, and both act through the same categorical interface so their contributions accumulate at each integration step.

Table V reports per-chunk inference latency measured on a workstation with an AMD Ryzen 7 5800X3D CPU and an NVIDIA RTX 5090 GPU. CFG evaluates the network twice per step, doubling the per-step DiT cost and accounting for the rise from 155 to 289 ms. The critic adds a smaller forward and backward pass. At 350 ms, the full configuration completes well within the 1.28 s spanned by each action chunk, and asynchronous inference computes the next chunk during execution, leaving margin for action chunk smoothing without bottlenecking execution.

A CFM policy with the same DiT backbone converges in training but fails all four tasks at evaluation. Rollout inspection reveals severe high-frequency oscillation in the commanded joint trajectories: Fig. 9 shows peak joint jerk on the order of 10^4 rad/s^3 , whereas VQActFlow remains near $2 \times 10^2 \text{ rad/s}^3$, roughly two orders of magnitude smoother. We attribute this to the VQ-VAE’s temporal compression

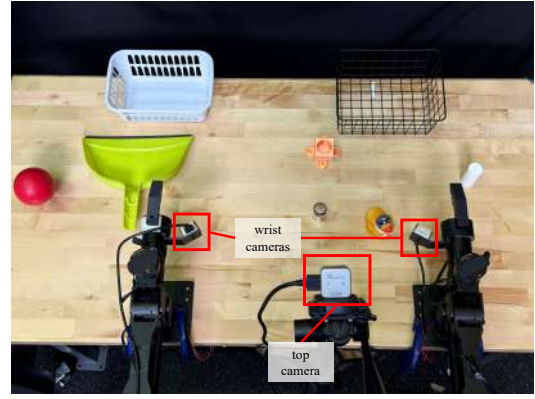


Fig. 7. Bimanual manipulation experimental setup with ALOHA-style arms and 3 cameras.

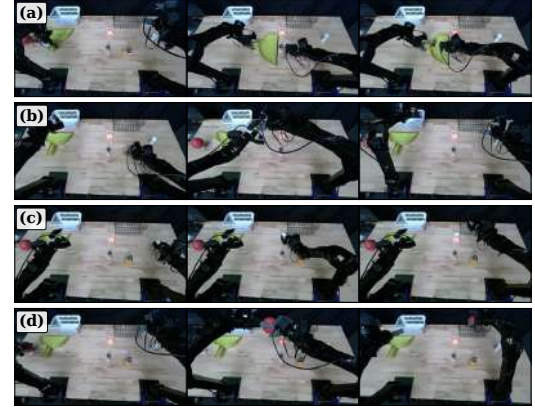


Fig. 8. Bimanual manipulation evaluation tasks. (a) Sweep the battery into the dust pan. (b) Place the rubber duck into the white bin. (c) Place the cylinder into the hole. (d) Place the red ball into the black bin.

structure: the decoder maps code sequences into action chunks through a learned compositional vocabulary that encodes inter-code dependencies including trajectory smoothness, while continuous flow matching at the same backbone capacity fails to produce smooth trajectories. Representative rollouts are included in the supplementary video.

VI. CONCLUSIONS

VQActFlow demonstrates that maintaining an explicit preference over discrete action modes during generation provides a more effective interface for inference-time guidance than continuous-embedding alternatives. At matched training conditions, the same CFG mechanism produces larger gains, peaks at lower guidance weights, and reaches higher ceilings than baselines across LIBERO-Goal, LIBERO-90, and the G1 and bimanual tasks. The contrastively-trained codebook critic operates through the same interface and supplies a feasibility signal that CFG alone does not. VQActFlow deploys successfully on two distinct hardware platforms, a Unitree G1 humanoid and an ALOHA-style bimanual system, executing contact-rich tasks where continuous flow matching at matched capacity fails.

Several directions remain open. The VQ-VAE bottleneck imposes an information loss floor that ultimately bounds policy performance, and more expressive action tokenization schemes may relax this constraint while preserving the categorical interface. A second direction is to use VFM as the

TABLE IV
SUCCESS RATES (%) ON BIMANUAL MANIPULATION TASKS

Method	Battery	Duck	Cylinder	Ball	Avg.
Discrete Policy, $w=1.0$	35.0	70.0	15.0	75.0	48.8
Discrete Policy, $w=4.0$	65.0	70.0	35.0	75.0	61.3
VQActFlow, $w=1.0, \lambda=0.0$	40.0	70.0	15.0	55.0	45.0
VQActFlow, $w=1.0, \lambda=1.0$	50.0	75.0	50.0	65.0	60.0
VQActFlow, $w=4.0, \lambda=0.0$	70.0	85.0	55.0	85.0	73.8
VQActFlow, $w=4.0, \lambda=1.0$	70.0	90.0	65.0	85.0	77.5

TABLE V
BIMANUAL INFERENCE TIME PER ACTION CHUNK (MS).

Configuration	Unguided	CFG	Critic	CFG + critic
Inference time	155.2	288.9	216.7	350.2

action head of a VLA model, where the categorical output naturally interfaces with language-model token vocabularies.

REFERENCES

- [1] H. Bharadhwaj, J. Vakil, M. Sharma, A. Gupta, S. Tulsiani, and V. Kumar, “Roboagent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking,” in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 4788–4795.
- [2] X. Ma, S. Patidar, I. Houghton, and S. James, “Hierarchical diffusion policy for kinematics-aware multi-task robotic manipulation,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 18 081–18 090.
- [3] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiuallah, and L. Pinto, “Behavior generation with latent actions,” in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 26 991–27 008.
- [4] Z. Zhao, S. Cheng, Y. Ding, Z. Zhou, S. Zhang, D. Xu, and Y. Zhao, “A survey of optimization-based task and motion planning: From classical to learning approaches,” *IEEE/ASME Trans. Mechatronics*, vol. 30, no. 4, pp. 2799–2825, 2024.
- [5] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *Int. J. Robot. Res.*, vol. 44, no. 10–11, pp. 1684–1704, 2025.
- [6] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat, “Flow matching guide and code,” *arXiv:2412.06264*, 2024.
- [7] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv:2410.24164*, 2024.
- [8] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv:2207.12598*, 2022.
- [9] P. Dhariwal and A. Nichol, “Diffusion models beat gans on image synthesis,” *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, pp. 8780–8794, 2021.
- [10] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 9902–9915.
- [11] A. Van Den Oord, O. Vinyals *et al.*, “Neural discrete representation learning,” *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [12] K. Wu, Y. Zhu, J. Li, J. Wen, N. Liu, Z. Xu, and J. Tang, “Discrete policy: Learning disentangled action space for multi-task robotic manipulation,” in *Proc. IEEE Int. Conf. Robot. Autom.*, 2025, pp. 8811–8818.
- [13] F. Eijkelboom, G. Bartosh, C. Andersson Naesseth, M. Welling, and J.-W. van de Meent, “Variational flow matching for graph generation,” *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 11 735–11 764, 2024.
- [14] R.-A. Mătișan, V. T. Hu, G. Bartosh, B. Ommer, C. G. Snoek, M. Welling, J.-W. van de Meent, M. M. Derakhshani, and F. Eijkelboom, “Purception: Variational flow matching for vector-quantized image generation,” *arXiv:2510.01478*, 2025.
- [15] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, pp. 6840–6851, 2020.
- [16] X. Huang, Y. Chi, R. Wang, Z. Li, X. B. Peng, S. Shao, B. Nikolic, and K. Sreenath, “Diffuselo: Real-time legged locomotion control with diffusion from offline datasets,” in *Proc. Conf. Robot Learn.*, 2025, pp. 1567–1589.

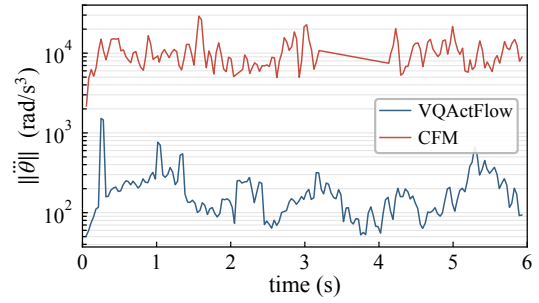


Fig. 9. Motion smoothness comparison between VQActFlow and CFM for bimanual manipulation hardware experiment.

- [17] S. H. Høeg, A. Vaaler, C. Liu, O. Egeland, and Y. Du, “Hybrid diffusion for simultaneous symbolic and continuous planning,” *IEEE Robot. Autom. Lett.*, 2026.
- [18] I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Chen, G. Synnaeve, Y. Adi, and Y. Lipman, “Discrete flow matching,” *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 133 345–133 385, 2024.
- [19] A. Campbell, J. Yim, R. Barzilay, T. Rainforth, and T. Jaakkola, “Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design,” in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 5453–5512.
- [20] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong *et al.*, “Openvla: An open-source vision-language-action model,” in *Proc. Conf. Robot Learn.*, 2025, pp. 2679–2713.
- [21] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, “Fast: Efficient action tokenization for vision-language-action models,” *arXiv:2501.09747*, 2025.
- [22] C. Liu, X. Han, J. Gao, Y. Zhao, H. Chen, and Y. Du, “Oat: Ordered action tokenization,” *arXiv:2602.04215*, 2026.
- [23] Q. Zheng, M. Le, N. Shaul, Y. Lipman, A. Grover, and R. T. Chen, “Guided flows for generative modeling and decision making,” *arXiv:2311.13443*, 2023.
- [24] W. Xiao, T.-H. Wang, C. Gan, R. Hasani, M. Lechner, and D. Rus, “Safediffuser: Safe planning with diffusion probabilistic models,” in *Proc. Int. Conf. Learn. Represent.*, 2023.
- [25] J.-H. Bastek, W. Sun, and D. Kochmann, “Physics-informed diffusion models,” in *Proc. Int. Conf. Learn. Represent.*, vol. 2025, 2025, pp. 3360–3385.
- [26] C. Pan, Z. Yi, G. Shi, and G. Qu, “Model-based diffusion for trajectory optimization,” *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 57 914–57 943, 2024.
- [27] P. Esser, R. Rombach, and B. Ommer, “Taming transformers for high-resolution image synthesis,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 12 873–12 883.
- [28] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 4195–4205.
- [29] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [30] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, “Learning transferable visual models from natural language supervision,” in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 8748–8763.
- [31] A. v. d. Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv:1807.03748*, 2018.
- [32] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Proc. Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 44 776–44 791, 2023.
- [33] R. Cadene, S. Alibert, A. Soare, Q. Galloudec, A. Zouitine, S. Palma, P. Kooijmans, M. Aractingi, M. Shukor, D. Aubakirova, M. Russi, F. Capuano, C. Pascal, J. Choghari, J. Moss, and T. Wolf, “Lerobot: State-of-the-art machine learning for real-world robotics in pytorch,” <https://github.com/huggingface/lerobot>, 2024.
- [34] Y. Ze, S. Zhao, W. Wang, A. Kanazawa, R. Duan, P. Abbeel, G. Shi, J. Wu, and C. K. Liu, “Twist2: Scalable, portable, and holistic humanoid data collection system,” *arXiv:2511.02832*, 2025.
- [35] Z. Zhao, L. Yu, K. Jing, and N. Yang, “Xrobotoolkit: A cross-platform framework for robot teleoperation,” in *Proc. IEEE/SICE Int. Symp. Syst. Integr.*, 2026, pp. 15–20.